


```
// Convert synchronized read Gray pointer to binary
always_comb begin
  rgray_sync_bin[PTR_WIDTH-1] = rgray_sync[PTR_WIDTH-1];
  for (int i = PTR_WIDTH - 2; i >= 0; i--) begin
    rgray_sync_bin[i] = rgray_sync_bin[i+1] ^ rgray_sync[i];
  end
end

//============================================================================
// Full Flag Generation
//============================================================================

// Full condition in Gray code:
// - MSB and MSB-1 are inverted
// - Remaining bits are equal
// Note: Generate block handles edge case for small FIFO depths (PTR_WIDTH <= 2)
generate
  if (PTR_WIDTH > 2) begin : gen_full_normal
    // Standard case: check top 2 bits inverted, rest equal
    assign wfull_next = (wgray_next[PTR_WIDTH-1]   != rgray_sync[PTR_WIDTH-1]) &&
                        (wgray_next[PTR_WIDTH-2]   != rgray_sync[PTR_WIDTH-2]) &&
                        (wgray_next[PTR_WIDTH-3:0] == rgray_sync[PTR_WIDTH-3:0]);
  end else begin : gen_full_small
    // Edge case for PTR_WIDTH == 2 (FIFO_DEPTH == 2, though not recommended)
    // Only check that top 2 bits are inverted
    assign wfull_next = (wgray_next[PTR_WIDTH-1:PTR_WIDTH-2] ==
                        ~rgray_sync[PTR_WIDTH-1:PTR_WIDTH-2]);
  end
endgenerate

//============================================================================
// Almost Full Flag Generation
//============================================================================

// Calculate fill level (approximate due to synchronization latency)
assign fill_level = wbin_next - rgray_sync_bin;

// Almost full when fill level reaches threshold
```

```
//============================================================================
// Almost Full Flag Generation
//============================================================================

// Calculate fill level (approximate due to synchronization latency)
assign fill_level = wbin_next - rgray_sync_bin;

// Almost full when fill level reaches threshold
assign walmost_full_next = (fill_level >= AF_THRESHOLD);

//============================================================================
// Overflow Detection
//============================================================================

// Overflow occurs when write is attempted while full
assign overflow_next = overflow_error | (wr_en & wfull);

//============================================================================
// Fill Count Output
//============================================================================

// Export fill level for external monitoring (approximate value)
assign wfill_count = fill_level[ADDR_WIDTH:0];

//============================================================================
// Sequential Logic
//============================================================================

always_ff @(posedge wclk or negedge wrst_n) begin
  if (!wrst_n) begin
    wbin           <= '0;
    wgray          <= '0;
    wfull          <= 1'b0;
    walmost_full   <= 1'b0;
    overflow_error <= 1'b0;
  end else begin
    wbin         <= wbin_next;
    wgray        <= wgray_next;
    wfull        <= wfull_next;
```

```
always_ff @(posedge wclk or negedge wrst_n) begin
  if (!wrst_n) begin
    wbin           <= '0;
    wgray          <= '0;
    wfull          <= 1'b0;
    walmost_full   <= 1'b0;
    overflow_error <= 1'b0;
  end else begin
    wbin         <= wbin_next;
    wgray        <= wgray_next;
    wfull        <= wfull_next;
    walmost_full <= walmost_full_next;
    // Overflow is sticky, cleared only by explicit clear signal
    overflow_error <= clr_overflow ? 1'b0 : overflow_next;
  end
end

//============================================================================
// Assertions
//============================================================================

// No write when full
// synthesis translate_off
property no_write_when_full;
  @(posedge wclk) disable iff (!wrst_n)
  (wfull && wr_en) |-> $stable(wbin);
endproperty
assert property (no_write_when_full) else
  $error("Write attempted when FIFO is full");

// Gray code single-bit transition
property gray_single_bit_change;
  @(posedge wclk) disable iff (!wrst_n)
  !$stable(wgray) |-> $countones(wgray ^ $past(wgray)) == 1;
endproperty
assert property (gray_single_bit_change) else
  $error("Gray pointer changed more than one bit");
// synthesis translate_on
```

async_fifo_pkg.sv | async_fifo.sv | fifo_mem.sv | read_ptr_handler.sv | reset_sync.sv | sync_ff.sv | **write_ptr_handler.sv**

```
  //============================================================================
  // Assertions
  //============================================================================

  // No write when full
  // synthesis translate_off
  property no_write_when_full;
    @(posedge wclk) disable iff (!wrst_n)
    (wfull && wr_en) |-> $stable(wbin);
  endproperty
  assert property (no_write_when_full) else
    $error("Write attempted when FIFO is full");

  // Gray code single-bit transition
  property gray_single_bit_change;
    @(posedge wclk) disable iff (!wrst_n)
    !$stable(wgray) |-> $countones(wgray ^ $past(wgray)) == 1;
  endproperty
  assert property (gray_single_bit_change) else
    $error("Gray pointer changed more than one bit");
  // synthesis translate_on

  // Parameter validation
  initial begin
    assert (ADDR_WIDTH >= 2) else
      $fatal(1, "ADDR_WIDTH must be at least 2");
    assert (AF_THRESHOLD > 0 && AF_THRESHOLD < DEPTH) else
      $fatal(1, "AF_THRESHOLD must be between 1 and DEPTH-1");
  end

endmodule : write_ptr_handler
~
~
~
~
```

```
//------------------------------------------------------------------------------
// File: read_ptr_handler.sv
// Description: Read pointer handler for asynchronous FIFO
//              - Manages binary and Gray-coded read pointer
//              - Generates empty and almost-empty flags
//              - Operates in read clock domain
//              - Provides fill count and underflow detection
//------------------------------------------------------------------------------

module read_ptr_handler #(
  parameter int ADDR_WIDTH   = 4,    // Address width (log2 of FIFO depth)
  parameter int AE_THRESHOLD = 2     // Almost empty threshold
)(
  input  logic                  rclk,           // Read clock
  input  logic                  rrst_n,         // Read domain reset (active low)
  input  logic                  rd_en,          // Read enable
  input  logic [ADDR_WIDTH:0]   wgray_sync,     // Synchronized write Gray pointer
  input  logic                  clr_underflow,  // Clear underflow flag

  output logic [ADDR_WIDTH-1:0] raddr,          // Read address for memory
  output logic [ADDR_WIDTH:0]   rgray,          // Gray-coded read pointer (for sync)
  output logic                  rempty,         // Empty flag
  output logic                  rempty_next_out,// Next empty state (for FWFT)
  output logic                  ralmost_empty,  // Almost empty flag
  output logic [ADDR_WIDTH:0]   rfill_count,    // Approximate fill level (read domain)
  output logic                  underflow_error // Sticky underflow error flag
);

  //============================================================================
  // Local Parameters
  //============================================================================

  localparam int PTR_WIDTH = ADDR_WIDTH + 1;  // Extra bit for wrap detection
  localparam int DEPTH     = 1 << ADDR_WIDTH;

  //============================================================================
  // Internal Signals
  //============================================================================
```

```
// Internal Signals
//============================================================================

logic [PTR_WIDTH-1:0] rbin;           // Binary read pointer
logic [PTR_WIDTH-1:0] rbin_next;      // Next binary read pointer
logic [PTR_WIDTH-1:0] rgray_next;     // Next Gray read pointer
logic [PTR_WIDTH-1:0] wgray_sync_bin; // Synchronized write pointer in binary
logic [PTR_WIDTH-1:0] fill_level;     // Current fill level

logic                 rempty_next;    // Next empty flag value
logic                 ralmost_empty_next;
logic                 underflow_next; // Next underflow flag value

//============================================================================
// Binary Pointer Logic
//============================================================================

// Increment read pointer when read is valid (not empty)
assign rbin_next = rbin + (rd_en & ~rempty);

// Memory address is lower bits of binary pointer
assign raddr = rbin[ADDR_WIDTH-1:0];

//============================================================================
// Binary to Gray Conversion
//============================================================================

// Convert next binary pointer to Gray code
assign rgray_next = rbin_next ^ (rbin_next >> 1);

//============================================================================
// Gray to Binary Conversion (for fill level calculation)
//============================================================================

// Convert synchronized write Gray pointer to binary
always_comb begin
  wgray_sync_bin[PTR_WIDTH-1] = wgray_sync[PTR_WIDTH-1];
  for (int i = PTR_WIDTH - 2; i >= 0; i--) begin
    wgray_sync_bin[i] = wgray_sync_bin[i+1] ^ wgray_sync[i];
```

```
// Convert synchronized write Gray pointer to binary
always_comb begin
  wgray_sync_bin[PTR_WIDTH-1] = wgray_sync[PTR_WIDTH-1];
  for (int i = PTR_WIDTH - 2; i >= 0; i--) begin
    wgray_sync_bin[i] = wgray_sync_bin[i+1] ^ wgray_sync[i];
  end
end

//============================================================================
// Empty Flag Generation
//============================================================================

// Empty condition: Gray-coded pointers are equal
assign rempty_next = (rgray_next == wgray_sync);

// Export next empty state for FWFT mode
assign rempty_next_out = rempty_next;

//============================================================================
// Almost Empty Flag Generation
//============================================================================

// Calculate fill level (approximate due to synchronization latency)
assign fill_level = wgray_sync_bin - rbin_next;

// Almost empty when fill level is at or below threshold
assign ralmost_empty_next = (fill_level <= AE_THRESHOLD);

//============================================================================
// Underflow Detection
//============================================================================

// Underflow occurs when read is attempted while empty
assign underflow_next = underflow_error | (rd_en & rempty);

//============================================================================
// Fill Count Output
//============================================================================
```

```
//============================================================================
// Fill Count Output
//============================================================================

// Export fill level for external monitoring (approximate value)
assign rfill_count = fill_level[ADDR_WIDTH:0];

//============================================================================
// Sequential Logic
//============================================================================

always_ff @(posedge rclk or negedge rrst_n) begin
  if (!rrst_n) begin
    rbin            <= '0;
    rgray           <= '0;
    rempty          <= 1'b1;  // Default to empty on reset
    ralmost_empty   <= 1'b1;  // Default to almost empty on reset
    underflow_error <= 1'b0;
  end else begin
    rbin          <= rbin_next;
    rgray         <= rgray_next;
    rempty        <= rempty_next;
    ralmost_empty <= ralmost_empty_next;
    // Underflow is sticky, cleared only by explicit clear signal
    underflow_error <= clr_underflow ? 1'b0 : underflow_next;
  end
end

//============================================================================
// Assertions
//============================================================================

// No read when empty
// synthesis translate_off
property no_read_when_empty;
  @(posedge rclk) disable iff (!rrst_n)
  (rempty && rd_en) |-> $stable(rbin);
endproperty
```

```
    end
  end

  //============================================================================
  // Assertions
  //============================================================================

  // No read when empty
  // synthesis translate_off
  property no_read_when_empty;
    @(posedge rclk) disable iff (!rrst_n)
    (rempty && rd_en) |-> $stable(rbin);
  endproperty
  assert property (no_read_when_empty) else
    $error("Read attempted when FIFO is empty");

  // Gray code single-bit transition
  property gray_single_bit_change;
    @(posedge rclk) disable iff (!rrst_n)
    !$stable(rgray) |-> $countones(rgray ^ $past(rgray)) == 1;
  endproperty
  assert property (gray_single_bit_change) else
    $error("Gray pointer changed more than one bit");
  // synthesis translate_on

  // Parameter validation
  initial begin
    assert (ADDR_WIDTH >= 2) else
      $fatal(1, "ADDR_WIDTH must be at least 2");
    assert (AE_THRESHOLD >= 0 && AE_THRESHOLD < DEPTH) else
      $fatal(1, "AE_THRESHOLD must be between 0 and DEPTH-1");
  end

endmodule : read_ptr_handler
~
~
~
~
~
```

```
//------------------------------------------------------------------------------
// File: fifo_mem.sv
// Description: True dual-port memory for asynchronous FIFO
//              Independent read and write ports with separate clocks
//              No arbitration needed - full/empty flags prevent conflicts
//              Optional FWFT (First-Word Fall-Through) mode support
//------------------------------------------------------------------------------

module fifo_mem #(
  parameter int  DATA_WIDTH = 32,    // Width of data bus
  parameter int  ADDR_WIDTH = 4,     // Address width (log2 of FIFO depth)
  parameter bit  FWFT_MODE  = 0,     // First-Word Fall-Through mode
  parameter bit  INIT_MEM   = 1      // Initialize memory to zero (simulation)
)(
  // Write Port (wclk domain)
  input  logic                  wclk,       // Write clock
  input  logic                  wr_en,      // Write enable
  input  logic [ADDR_WIDTH-1:0] waddr,      // Write address
  input  logic [DATA_WIDTH-1:0] wdata,      // Write data

  // Read Port (rclk domain)
  input  logic                  rclk,       // Read clock
  input  logic                  rd_en,      // Read enable
  input  logic [ADDR_WIDTH-1:0] raddr,      // Read address
  output logic [DATA_WIDTH-1:0] rdata,      // Read data

  // FWFT Control (only used when FWFT_MODE=1)
  input  logic                  rempty,     // Empty flag for FWFT bypass
  input  logic                  rempty_next // Next empty state for FWFT
);

  //============================================================================
  // Memory Array
  //============================================================================

  // FIFO depth derived from address width
  localparam int DEPTH = 1 << ADDR_WIDTH;

  // Memory storage array
```




```
// Memory Array
//==============================================================================

// FIFO depth derived from address width
localparam int DEPTH = 1 << ADDR_WIDTH;

// Memory storage array
logic [DATA_WIDTH-1:0] mem [0:DEPTH-1];

// FWFT bypass register
logic [DATA_WIDTH-1:0] rdata_reg;
logic [DATA_WIDTH-1:0] rdata_bypass;
logic                  fwft_bypass_en;

//==============================================================================
// Memory Initialization (for simulation - prevents X propagation)
//==============================================================================

// synthesis translate_off
generate
  if (INIT_MEM) begin : gen_mem_init
    initial begin
      for (int i = 0; i < DEPTH; i++) begin
        mem[i] = '0;
      end
    end
  end
endgenerate
// synthesis translate_on

//==============================================================================
// Write Port Logic
//==============================================================================

// Note: Using 'always' instead of 'always_ff' to allow simulation
// initialization of mem array without ICPD errors
always @(posedge wclk) begin
  if (wr_en) begin
    mem[waddr] <= wdata;
```

```
// Note: Using 'always' instead of 'always_ff' to allow simulation
// initialization of mem array without ICPD errors
always @(posedge wclk) begin
  if (wr_en) begin
    mem[waddr] <= wdata;
  end
end

//============================================================================
// Read Port Logic
//============================================================================

generate
  if (FWFT_MODE) begin : gen_fwft
    //------------------------------------------------------------------------
    // FWFT Mode: Combinational bypass when transitioning from empty
    //------------------------------------------------------------------------

    // Registered read data
    always_ff @(posedge rclk) begin
      if (rd_en || fwft_bypass_en) begin
        rdata_reg <= mem[raddr];
      end
    end

    // FWFT bypass enable: when FIFO transitions from empty to non-empty
    // Data should appear on rdata without requiring rd_en
    assign fwft_bypass_en = rempty && !rempty_next;

    // Combinational bypass path for first word
    assign rdata_bypass = mem[raddr];

    // Output mux: use bypass when transitioning from empty, else registered
    assign rdata = fwft_bypass_en ? rdata_bypass : rdata_reg;

  end else begin : gen_standard
    //------------------------------------------------------------------------
    // Standard Mode: Registered read output for timing closure
    //------------------------------------------------------------------------
```

```
    // Output mux: use bypass when transitioning from empty, else registered
    assign rdata = fwft_bypass_en ? rdata_bypass : rdata_reg;

  end else begin : gen_standard
    //------------------------------------------------------------------------
    // Standard Mode: Registered read output for timing closure
    //------------------------------------------------------------------------

    always_ff @(posedge rclk) begin
      if (rd_en) begin
        rdata_reg <= mem[raddr];
      end
    end

    assign rdata = rdata_reg;

    // Unused signals in standard mode
    assign fwft_bypass_en = 1'b0;
    assign rdata_bypass = '0;
  end
endgenerate

//============================================================================
// Assertions
//============================================================================

// Parameter validation
initial begin
  assert (DATA_WIDTH >= 1) else
    $fatal(1, "DATA_WIDTH must be at least 1");
  assert (ADDR_WIDTH >= 1) else
    $fatal(1, "ADDR_WIDTH must be at least 1");
end

endmodule : fifo_mem
~
~
~
```